SFTP API Reference

SFTP Enterprise Management System – API Reference

Revision: 1 **Last modified:** 2026-07-11T23:00:00Z **Audience:** Developers integrating with the SFTP management REST API. **API version:** 0.1.0-dev **Live verification:** 10/10 MANUAL-QA PASS (commit cd16d77, 2026-07-11).

Table of contents

---

1. Base URL

All endpoints are served under:

```
http://127.0.0.1:7722/api/v1
```

The API binds **127.0.0.1** by default. In production, place a TLS-terminating reverse proxy (nginx, Caddy, HAProxy) in front of the API, or set `API_BIND` to the appropriate interface.

---

2. Authentication

The API uses **JWT Bearer tokens** (HS256) signed with a server-side secret.

2.1 Obtaining a token pair

Send a `POST /auth/login` with the super-admin username and password. On success the response carries:

Field	Type	Meaning
`access_token`	string	Short-lived JWT (default 15 minutes).
`refresh_token`	string	Long-lived JWT (default 7 days).
`token_type`	string	Always `"Bearer"`.
`expires_in`	integer	Access token TTL in seconds.
`refresh_expires_in`	integer	Refresh token TTL in seconds.

2.2 Using the access token

Every secured endpoint requires:

```
Authorization: Bearer <access_token>
```

Missing, malformed, or expired tokens produce a `401 Unauthorized` response.

2.3 Refreshing tokens

When the access token expires, call `POST /auth/refresh` with the refresh token to obtain a **brand-new pair**. The old refresh token is NOT revoked during refresh – a single admin session may hold multiple valid refresh tokens.

2.4 Token lifecycle

```
Login  ──>  access (15m)  +  refresh (7d)
                 │
                 ▼ (access expires)
```

```
          POST /auth/refresh  ──>  new access (15m)  +  new
refresh (7d)
                │
                ▼ (admin logs out)
          POST /auth/logout   ──>  that refresh token revoked
                                   (cannot be used for refresh
again)
```

Token TTLs are configurable via `ACCESS_TOKEN_TTL` and `REFRESH_TOKEN_TTL` environment variables (defaults: 15m / 168h).

2.5 Token kinds are enforced

Access tokens carry `"kind":"access"`; refresh tokens carry `"kind":"refresh"`. An access token cannot be used on `/auth/refresh`, and a refresh token cannot be used on secured endpoints – the server rejects each with `401`.

---

3. Rate limiting

Login and refresh endpoints (`POST /auth/login`, `POST /auth/refresh`) are rate-limited per client IP using a fixed-window counter (10 requests per minute by default). On exhaustion the server returns `429 Too Many Requests` with a `Retry-After: 60` header.

Configurable via `LOGIN_RATE_LIMIT` and `LOGIN_RATE_WINDOW` env vars. Set `LOGIN_RATE_LIMIT=0` to disable (test mode only – never in production).

---

4. Error envelope

Every error response uses the same JSON shape:

```json
{
  "code": "unauthorized",
  "error": "invalid username or password"
}
```

Field	Type	Meaning
`code`	string	Machine-readable error code (see Section 8).
`error`	string	Human-readable message – safe to display.

Passwords and secrets **never** appear in error responses.

---

5. Endpoint reference

5.1 GET /health

Public liveness probe. No authentication required.

Request

```
GET /api/v1/health
```

Response `200 OK`

```
{
  "status": "ok",
  "version": "0.1.0-dev",
  "time": "2026-07-11T22:33:48Z",
  "firebase": "disabled"
}
```

Field	Type	Values
`status`	string	Always `"ok"`.
`version`	string	API version from `API_VERSION` env (default `0.1.0-dev`).
`time`	string	Current UTC time in RFC 3339.
`firebase`	string	`"disabled"`, `"unavailable"`, `"connected"`, or `"unhealthy"`.

Example (MANUAL-QA confirmed)

```
curl -s http://127.0.0.1:7722/api/v1/health | jq
```

```
{
  "status": "ok",
  "version": "0.1.0-dev",
  "time": "2026-07-11T22:33:48Z",
  "firebase": "disabled"
}
```

5.2 POST /auth/login

Authenticate as super-admin. Returns a JWT access + refresh token pair.

Request

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{
  "username": "admin",
  "password": "<superadmin-password>"
}
```

Field	Type	Required	Notes
`username`	string	yes	Super-admin username (default `admin`).
`password`	string	yes	Super-admin password from `SUPERADMIN_PASSWORD` env.

Response `200 OK`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_expires_in": 604800
}
```

Errors

Status	Code	When
400	`bad_request`	Invalid JSON body.
400	`validation_error`	Username or password missing.
401	`invalid_credentials`	Wrong username or password (identical message for both – no user enumeration).
429	`rate_limited`	Too many login attempts.
500	`internal_error`	

Status	Code	When
		Store error, JWT signing failure.

Example (MANUAL-QA confirmed)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"<REDACTED>"}' | jq
```

Wrong password yields:

```
{
  "code": "invalid_credentials",
  "error": "invalid username or password"
}
```

5.3 POST /auth/refresh

Exchange a valid refresh token for a new JWT pair. Also rate-limited.

Request

```
POST /api/v1/auth/refresh
Content-Type: application/json

{
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Field	Type	Required	Notes
`refresh_token`	string	yes	A valid (unrevoked, unexpired) refresh token.

Response `200 OK`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_expires_in": 604800
}
```

Errors

Status	Code	When
400	`bad_request`	Invalid JSON body.
400	`validation_error`	`refresh_token` missing.
401	`unauthorized`	Token expired, wrong kind, or revoked.
429	`rate_limited`	Too many refresh attempts.
500	`internal_error`	JWT signing failure.

Example (MANUAL-QA confirmed – logout then rejected refresh)

```
# Refresh succeeds with a valid token
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/refresh \
  -H 'Content-Type: application/json' \
  -d '{"refresh_token":"eyJ..."}' | jq '.expires_in'
# 900

# After logout, the same refresh token is rejected
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/logout \
  -H 'Authorization: Bearer <access>' \
  -H 'Content-Type: application/json' \
  -d '{"refresh_token":"eyJ..."}'

curl -s -X POST http://127.0.0.1:7722/api/v1/auth/refresh \
  -H 'Content-Type: application/json' \
  -d '{"refresh_token":"eyJ..."}' | jq
# { "code": "unauthorized", "error": "invalid or expired refresh
        token" }
```

5.4 GET /auth/me

Return the identity of the caller from the bearer token.

Request

```
GET /api/v1/auth/me
Authorization: Bearer <access_token>
```

Response `200 OK`

```
{
  "username": "admin",
  "expires_at": "2026-07-11T22:48:48Z"
}
```

Field	Type	Meaning
`username`	string	

Field	Type	Meaning
		JWT subject – the super-admin username.
`expires_at`	string	ISO 8601 UTC expiry of the current access token.

Errors

Status	Code	When
401	`unauthorized`	Missing or invalid bearer token.

Example (MANUAL-QA confirmed)

```
curl -s http://127.0.0.1:7722/api/v1/auth/me \
  -H 'Authorization: Bearer <access_token>' | jq
# { "username": "admin", "expires_at": "2026-07-11T22:48:48Z" }
```

5.5 POST /auth/logout

Revoke one refresh token so it can no longer be used.

Request

```
POST /api/v1/auth/logout
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Field	Type	Required	Notes
`refresh_token`	string	yes	The refresh token to revoke.

Response `200 OK`

```
{
  "message": "logged out"
}
```

Errors

Status	Code	When
400	`bad_request`	Invalid JSON body.
400	`validation_error`	

Status	Code	When
		`refresh_token` missing.
401	`unauthorized`	Invalid/expired token, or missing access token.

Example (MANUAL-QA confirmed)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/logout \
  -H 'Authorization: Bearer <access_token>' \
  -H 'Content-Type: application/json' \
  -d '{"refresh_token":"<refresh>"}' | jq
# { "message": "logged out" }
```

5.6 GET /accounts

List all SFTP accounts. Password material is NEVER returned.

Request

```
GET /api/v1/accounts
Authorization: Bearer <access_token>
```

Response `200 OK`

```
{
  "accounts": [
    {
      "username": "testuser",
      "permission": "read_write",
      "uid": null,
      "gid": null,
      "home_dir": "/testuser",
      "enabled": true,
      "created_at": "2026-07-11T22:33:48Z",
      "updated_at": "2026-07-11T22:33:48Z"
    }
  ],
  "count": 1
}
```

Errors

Status	Code	When
401	`unauthorized`	Missing or invalid bearer token.
500	`internal_error`	Database error.

Example (MANUAL-QA confirmed)

```
curl -s http://127.0.0.1:7722/api/v1/accounts \
  -H 'Authorization: Bearer <access_token>' | jq '.count'
# 1
```

5.7 POST /accounts

Create a new SFTP account.

Request

```
POST /api/v1/accounts
Authorization: Bearer <access_token>
Content-Type: application/json
```

```
{
  "username": "newuser",
  "password": "secure-password",
  "permission": "read_write",
  "public_acknowledged": false,
  "uid": null,
  "gid": null,
  "home_dir": "",
  "enabled": true
}
```

Field	Type	Required	Default	Notes
`username`	string	**yes**	–	Must match `^[a-z_][a-z0-9_-]{0,31}$` (Linux-safe, max 32).
`password`	string	**yes**	–	Plaintext – bcrypt-hashed in memory, never echoed.
`permission`	string	no	`read_only`	One of `read_only`, `read_write`, `public`.
`public_acknowledged`	bool	no	`false`	**Must be `true`** when

Field	Type	Required	Default	Notes
				`permission` is `public`.
`uid`	integer	no	`null`	Optional POSIX UID for the chroot.
`gid`	integer	no	`null`	Optional POSIX GID for the chroot.
`home_dir`	string	no	`"/<username>"`	Must start with `/` if provided.
`enabled`	bool	no	`true`	Disabled accounts cannot connect.

Response `201 Created`

```
{
  "username": "newuser",
  "permission": "read_write",
  "uid": null,
  "gid": null,
  "home_dir": "/newuser",
  "enabled": true,
  "created_at": "2026-07-11T22:33:48Z",
  "updated_at": "2026-07-11T22:33:48Z"
}
```

Errors

Status	Code	When
400	`bad_request`	Invalid JSON body.
400	`validation_error`	Username/password missing, invalid username pattern, invalid permission, invalid home_dir.
401	`unauthorized`	Missing or invalid bearer token.
409	`conflict`	Username already exists.

Status	Code	When
422	`public_access_not_acknowledged`	`permission` is `public` but `public_acknowledged` is not `true`.
500	`internal_error`	Database error, hashing failure, vault error.

Example – create a public account WITH acknowledgment (MANUAL-QA confirmed)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \
  -H 'Authorization: Bearer <access_token>' \
  -H 'Content-Type: application/json' \
  -d '{"username":"pubuser","password":"pw","permission":"public","public_acknowle
        jq '.permission'
# "public"
```

Example – create a public account WITHOUT acknowledgment (MANUAL-QA confirmed)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \
  -H 'Authorization: Bearer <access_token>' \
  -H 'Content-Type: application/json' \
  -d '{"username":"pubuser2","password":"pw","permission":"public"}' |
        jq

{
  "code": "public_access_not_acknowledged",
  "error": "public access is never a default: set
        public_acknowledged=true to confirm"
}
```

5.8 GET /accounts/:username

Retrieve a single account.

Request

```
GET /api/v1/accounts/:username
Authorization: Bearer <access_token>
```

Response `200 OK`

```
{
  "username": "newuser",
  "permission": "read_write",
  "uid": null,
  "gid": null,
  "home_dir": "/newuser",
```

```
  "enabled": true,
  "created_at": "2026-07-11T22:33:48Z",
  "updated_at": "2026-07-11T22:33:48Z"
}
```

Errors

Status	Code	When
401	`unauthorized`	Missing or invalid bearer token.
404	`not_found`	No account with that username.
500	`internal_error`	Database error.

```
curl -s http://127.0.0.1:7722/api/v1/accounts/newuser \
  -H 'Authorization: Bearer <access_token>' | jq '.permission'
# "read_write"
```

5.9 PUT /accounts/:username

Update an existing account. The username in the URL path is authoritative; any `username` field in the body is replaced with the path value.

Partial update semantics: every field in the body is applied. Omit a field to preserve its current value. The `password` field is special: when omitted the existing hash is kept; when provided it is re-hashed.

Request

```
PUT /api/v1/accounts/:username
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "permission": "read_only",
  "enabled": true
}
```

Field	Type	Required	Default	Notes
`username`	string	no	Path value	Ignored if provided – path username wins.
`password`	string	no	(unchanged)	Providing a non-empty

Field	Type	Required	Default	Notes
				value re-hashes.
`permission`	string	no	`read_only`	One of `read_only`, `read_write`, `public`.
`public_acknowledged`	bool	no	`false`	Same guard as create.
`uid`	integer	no	(unchanged)	
`gid`	integer	no	(unchanged)	
`home_dir`	string	no	(unchanged)	
`enabled`	bool	no	(unchanged)	

Response `200 OK`

```
{
  "username": "newuser",
  "permission": "read_only",
  "uid": null,
  "gid": null,
  "home_dir": "/newuser",
  "enabled": true,
  "created_at": "2026-07-11T22:33:48Z",
  "updated_at": "2026-07-11T22:34:00Z"
}
```

Errors

Status	Code	When
400	`bad_request`	Invalid JSON body.
400	`validation_error`	Invalid field value.
401	`unauthorized`	Missing or invalid bearer token.
404	`not_found`	Account does not exist.
422	`public_access_not_acknowledged`	Permission changed to `public` without ack.
500	`internal_error`	

Status	Code	When
		Database error, hashing failure, vault error.

Example – change permission (MANUAL-QA confirmed)

```
curl -s -X PUT http://127.0.0.1:7722/api/v1/accounts/testuser \
  -H 'Authorization: Bearer <access_token>' \
  -H 'Content-Type: application/json' \
  -d '{"permission":"read_only"}' | jq '.permission'
# "read_only"
```

5.10 DELETE /accounts/:username

Permanently remove an SFTP account and its vault entry.

Request

```
DELETE /api/v1/accounts/:username
Authorization: Bearer <access_token>
```

Response `204 No Content`

Empty body. The `username` can no longer be looked up.

Errors

Status	Code	When
401	`unauthorized`	Missing or invalid bearer token.
404	`not_found`	Account does not exist.
500	`internal_error`	Database error.

Example (MANUAL-QA confirmed)

```
curl -s -o /dev/null -w '%{http_code}' \
  -X DELETE http://127.0.0.1:7722/api/v1/accounts/testuser \
  -H 'Authorization: Bearer <access_token>'
# 204
```

5.11 POST /sync

Render the `users.conf` file from the current account set in the database.

Request

```
POST /api/v1/sync
Authorization: Bearer <access_token>
```

Response `200 OK`

```
{
  "rendered_accounts": 1,
  "path": "data/users.conf"
}
```

Field	Type	Meaning
`rendered_accounts`	integer	Number of enabled accounts rendered.
`path`	string	Filesystem path of the generated `users.conf`.

Errors

Status	Code	When
401	`unauthorized`	Missing or invalid bearer token.
500	`internal_error`	Database or file-write error.

Example (MANUAL-QA confirmed)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/sync \
  -H 'Authorization: Bearer <access_token>' | jq
# { "rendered_accounts": 1, "path": "data/users.conf" }
```

The rendered `users.conf` follows the `atmoz/sftp` format. After sync, the SFTP container picks up the new configuration.

---

6. Account model reference

```
{
  "username": "jdoe",
  "permission": "read_write",
  "uid": 1001,
  "gid": 1001,
  "home_dir": "/jdoe",
  "enabled": true,
  "created_at": "2026-07-11T22:00:00Z",
  "updated_at": "2026-07-11T22:00:00Z"
}
```

Field	Type	Constraints	Default	Writable
`username`	string	`^[a-z_][a-z0-9_-]`	(required)	Create only

Field	Type	Constraints	Default	Writable
		{0,31}$, unique, max 32 chars		
permission	string	Closed enum: read_only, read_write, public	read_only	Yes
uid	integer	>=0, nullable	null (auto-assigned)	Yes
gid	integer	>=0, nullable	null (auto-assigned)	Yes
home_dir	string	Absolute path, must start with /	"/<username>"	Yes
enabled	bool	true/false	true	Yes
created_at	string	ISO 8601 UTC, read-only	Server-set	No
updated_at	string	ISO 8601 UTC, read-only	Server-set	No

6.1 Username rules

- Lowercase letters, digits, underscores, hyphens.
- Must start with a letter or underscore.
- Max 32 characters.
- Valid examples: jdoe, ftp_bot, backup-01, _service.
- Invalid: JohnDoe (uppercase), -ftp (starts with hyphen), a@b (special char).

6.2 Password handling

- The password is write-only. It never appears in any response.
- The API stores a bcrypt hash (cost 12) for admin authentication.
- The SFTP container receives a separate sha512-crypt (6) hash via the vault and users.conf.
- On create/update, both hashes are computed from the single plaintext in the request and the plaintext is immediately discarded.

6.3 Home directory layout

The sync layer provisions the pattern required by `atmoz/sftp` (chroot-safe):

```
/<username>/           (root-owned, non-writable -- chroot anchor)
/<username>/uploads/   (account-owned, writable -- where the user
drops files)
```

---

7. Permission enum reference

Value	SFTP behaviour	API guard	Use case
`read_only`	Download only; upload denied.	None. Default permission.	Read-only mirrors, anonymous distribution.
`read_write`	Full upload + download.	None.	Interactive users, backup targets.
`public`	Like `read_only` but over SSH without a password.	**`public_acknowledged=true` required.** Never a default.	Public file distribution – explicit admin opt-in.

The `public` permission is **never a default**. When a create or update request sets `permission` to `public` without `"public_acknowledged": true`, the API rejects the request with:

```
422 Unprocessable Entity
{
  "code": "public_access_not_acknowledged",
  "error": "public access is never a default: set
public_acknowledged=true to confirm"
}
```

This guard exists because a public account exposes files to anyone who can reach port 7721 – a high-impact security decision that must be deliberate.

---

8. Error codes

Stable wire values. Integrations may match on `code`.

HTTP	`code`	Meaning
400	`bad_request`	Request body is not valid JSON.
400	`validation_error`	A field failed server-side validation (missing required, invalid pattern, bad enum value).
401	`invalid_credentials`	Login: wrong username or password.
401	`unauthorized`	Missing/invalid/expired bearer token, or refresh token rejected.
404	`not_found`	Resource (account) does not exist.
409	`conflict`	Username already exists on create.
422	`public_access_not_acknowledged`	Permission `public` set without `public_acknowledged=true`.
429	`rate_limited`	Too many login/refresh requests from this IP.
500	`internal_error`	Server-side failure (database, hashing, token signing, file I/O).

All `5xx` errors indicate a server problem – retry with backoff. All `4xx` errors indicate a client problem – fix the request before retrying.

Sources verified (2026-07-11)

- Internal: `api/cmd/sftp-api/main.go`, `api/internal/api/router.go`, `api/internal/api/handlers_auth.go`, `api/internal/api/handlers_accounts.go`, `api/internal/api/handlers_sync.go`, `api/internal/api/errors.go`, `api/internal/store/store.go`, `api/internal/authn/authn.go`, `api/internal/config/config.go`.
- Live test evidence: `qa/results/MANUAL-QA-report.md` (10/10 PASS, commit cd16d77, 2026-07-11).